



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	SW-AyLP-APE-01: Lenguajes de Programación
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Primero - B
Alumnos participantes:	Casillas Ochoa Antony Sebastián
Asignatura:	Algoritmos y lógica de programación
Docente:	Ing. Jose Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Conocer los lenguajes de programación de la actualidad y sus antecesores.

Específicos:

- Identificar los principales lenguajes de programación que surgieron a lo largo de su evolución y reconocer sus aportes al desarrollo de la informática.
- Analizar las cinco generaciones de lenguajes de programación y sus principales características.
- Diferenciar los principales paradigmas de programación, como la programación procedural, orientada a objetos, funcional y declarativa.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 2

No presenciales: 0

2.4 Instrucciones

- El trabajo se desarrollará de manera individual.
- Realice una consulta sobre LA HISTORIA Y EVOLUCIÓN DE LOS LENGUAJES DE PROGRAMACIÓN.
- Seleccionar una herramienta que permita realizar infografías.
- Realizar una infografía a manera de resumen de la lectura realizada.
- Subir el documento al aula virtual en el formato de Guías APE en PDF

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial, TAC
- TAC - Computador o laptop
- Plataforma educativa de la UTA
- Internet para consultas (fuentes confiables)

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☒ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☒ Inteligencia Artificial
- Otros (Especifique): NA



2.6 Actividades por desarrollar

Desarrollar la guía APE en base al formato y las instrucciones brindadas

Historia y evolución de los lenguajes de programación

Un corte transversal por ocho décadas de código: de las primeras instrucciones en máquina a los lenguajes multiparadigma que compilan el software actual.

Introducción

Un lenguaje de programación es el puente entre el pensamiento lógico humano y las instrucciones que ejecuta una máquina. Su historia no es una lista de nombres: es un proceso evolutivo, donde cada generación resuelve las limitaciones de la anterior y hereda parte de sus sintaxis, ideas y sus errores. Igual que en un corte geológico, las capas más profundas — el código máquina y el ensamblador — siguen sosteniendo todo lo que se construyó encima. A continuación, un recorrido estrato por estrato.

1943 – 1953 | Código máquina

Principales acontecimientos:

- **Plankalkül – 1948**
- **Ensamblador – 1949**

Konrad Zuse diseñó **Plankalkül**, considerado uno de los primeros lenguajes de programación de alto nivel y el primero en incorporar estructuras lógicas. Sin embargo, no llegó a implementarse en su época.

Durante estos primeros años, las computadoras se programaban directamente mediante **código binario o lenguaje máquina**, utilizando instrucciones específicas para cada procesador. Posteriormente apareció el **lenguaje ensamblador**, que empleaba instrucciones mnemotécnicas asociadas directamente con el hardware.

1957 – 1964 | Primeros lenguajes de alto nivel

Principales acontecimientos:

- **FORTRAN – 1957**
- **LISP – 1958**
- **COBOL – 1959**
- **BASIC – 1964**

Durante este período surgió el concepto de **lenguaje de alto nivel**, permitiendo a los programadores escribir instrucciones mediante fórmulas, palabras y estructuras más cercanas al lenguaje humano.

FORTRAN se destacó en el cálculo científico; **COBOL**, en aplicaciones empresariales y administrativas; mientras que **LISP** introdujo importantes conceptos relacionados con la recursión y el procesamiento simbólico, fundamentales posteriormente para el desarrollo de la inteligencia artificial. **BASIC** facilitó el aprendizaje de la programación.

1970 – 1972 | Estructura y control

Principales acontecimientos:

- **Pascal – 1970**
- **C – 1972**
- **Smalltalk – 1972**

La llamada **crisis del software** impulsó la búsqueda de mejores métodos para organizar y desarrollar programas. Como respuesta, tomó fuerza la **programación estructurada**.

Pascal se convirtió en un lenguaje muy utilizado para la enseñanza de programación. **C** proporcionó un equilibrio entre el control del hardware y la portabilidad, mientras que **Smalltalk** desarrolló de manera profunda el paradigma de **programación orientada a objetos**, basado en objetos y clases.

1983 – 1985 | Objetos en producción

Principales acontecimientos:

- **C++ – 1983**
- **Ada – 1983**
- **Objective-C – 1984**



En esta etapa, la **programación orientada a objetos** comenzó a adquirir mayor importancia en la industria.

Bjarne Stroustrup desarrolló C++ a partir del lenguaje C, incorporando características como clases y herencia. Esto permitió crear sistemas de mayor complejidad manteniendo un alto rendimiento.

Lenguajes como **Ada** y **Objective-C** también contribuyeron al desarrollo de software de mayor escala y a la expansión de diferentes enfoques de programación.

1991 – 1995 | Explosión de Internet

Principales acontecimientos:

- **Python – 1991**
- **Java – 1995**
- **JavaScript – 1995**
- **PHP – 1995**

El crecimiento de Internet generó nuevas necesidades para el desarrollo de aplicaciones y páginas web.

Python destacó por su sintaxis sencilla y legible. **Java** popularizó la idea de “*escribir una vez y ejecutar en cualquier lugar*”, gracias a su máquina virtual.

Por otro lado, **JavaScript** permitió incorporar interactividad a las páginas web desde el navegador, mientras que **PHP** se convirtió en una herramienta importante para el desarrollo web del lado del servidor.

2000 – 2009 | Plataformas y computación en la nube

Principales acontecimientos:

- **C# – 2000**
- **Scala – 2004**
- **Go – 2009**

Microsoft desarrolló **C#** como parte de su plataforma **.NET**, ofreciendo una alternativa moderna para el desarrollo de aplicaciones.

Posteriormente aparecieron lenguajes como **Scala** y **Go**, orientados a responder a las necesidades de sistemas modernos, distribuidos y concurrentes.

Go, creado en Google, se caracterizó por su simplicidad, rapidez de compilación y eficiencia para aplicaciones y servidores de gran escala.

2010 – Actualidad | Seguridad y multiplataforma

Principales acontecimientos:

- **Rust – 2010**
- **Kotlin – 2011**
- **TypeScript – 2012**
- **Swift – 2014**

La programación moderna busca combinar **rendimiento, seguridad, productividad y compatibilidad entre diferentes plataformas**.

Rust ofrece un alto rendimiento junto con mecanismos de seguridad de memoria. **TypeScript** incorpora tipado estático sobre JavaScript, facilitando el desarrollo de aplicaciones grandes.

Kotlin se integró ampliamente con el ecosistema Android y la plataforma Java, mientras que **Swift** fue diseñado para el desarrollo de aplicaciones dentro del ecosistema de Apple.

Cinco generaciones de lenguajes de programación

1GL – Primera generación

Lenguaje máquina

Es el nivel más básico de programación. Utiliza instrucciones directamente representadas en código binario y está estrechamente ligado al procesador.

Característica principal: traducción directa al hardware.

2GL – Segunda generación

Lenguaje ensamblador



Utiliza **mnemónicos** para representar las instrucciones del procesador, haciendo que el código sea más comprensible que el lenguaje máquina.

Característica principal: mantiene una relación muy cercana con el hardware.

3GL – Tercera generación

Lenguajes de alto nivel

Utilizan una sintaxis más cercana al lenguaje humano y permiten desarrollar programas complejos con mayor facilidad.

Ejemplos: C, Java y Python.

Característica principal: mayor abstracción respecto al hardware.

4GL – Cuarta generación

Lenguajes declarativos

El programador especifica **qué resultado desea obtener**, mientras que el lenguaje o sistema se encarga de determinar cómo obtenerlo.

Ejemplo: SQL.

Característica principal: reducción de la cantidad de instrucciones necesarias para resolver un problema.

5GL – Quinta generación

Lenguajes basados en reglas

Se basan en **reglas, restricciones lógicas y sistemas de inferencia**, buscando que el sistema determine soluciones a partir de la información proporcionada.

Ejemplos: Prolog y aplicaciones relacionadas con inteligencia artificial.

Característica principal: utilización de lógica, reglas e inferencia para resolver problemas.

Cuatro formas de pensar el código

1. Programación procedural

Ejemplos: C, Pascal y FORTRAN.

El programa se organiza como una **secuencia de instrucciones y procedimientos** que modifican el estado del programa. Las tareas se dividen en funciones o procedimientos para facilitar su organización.

2. Programación orientada a objetos

Ejemplos: Java, C++ y Python.

Los datos y comportamientos se agrupan en **objetos**, los cuales interactúan entre sí. Este paradigma permite organizar programas complejos mediante conceptos como clases, objetos, herencia y encapsulamiento.

3. Programación funcional

Ejemplos: LISP y Haskell.

Se basa en la **composición de funciones**, buscando reducir la modificación del estado y trabajar, en la medida de lo posible, con funciones puras.

4. Programación declarativa

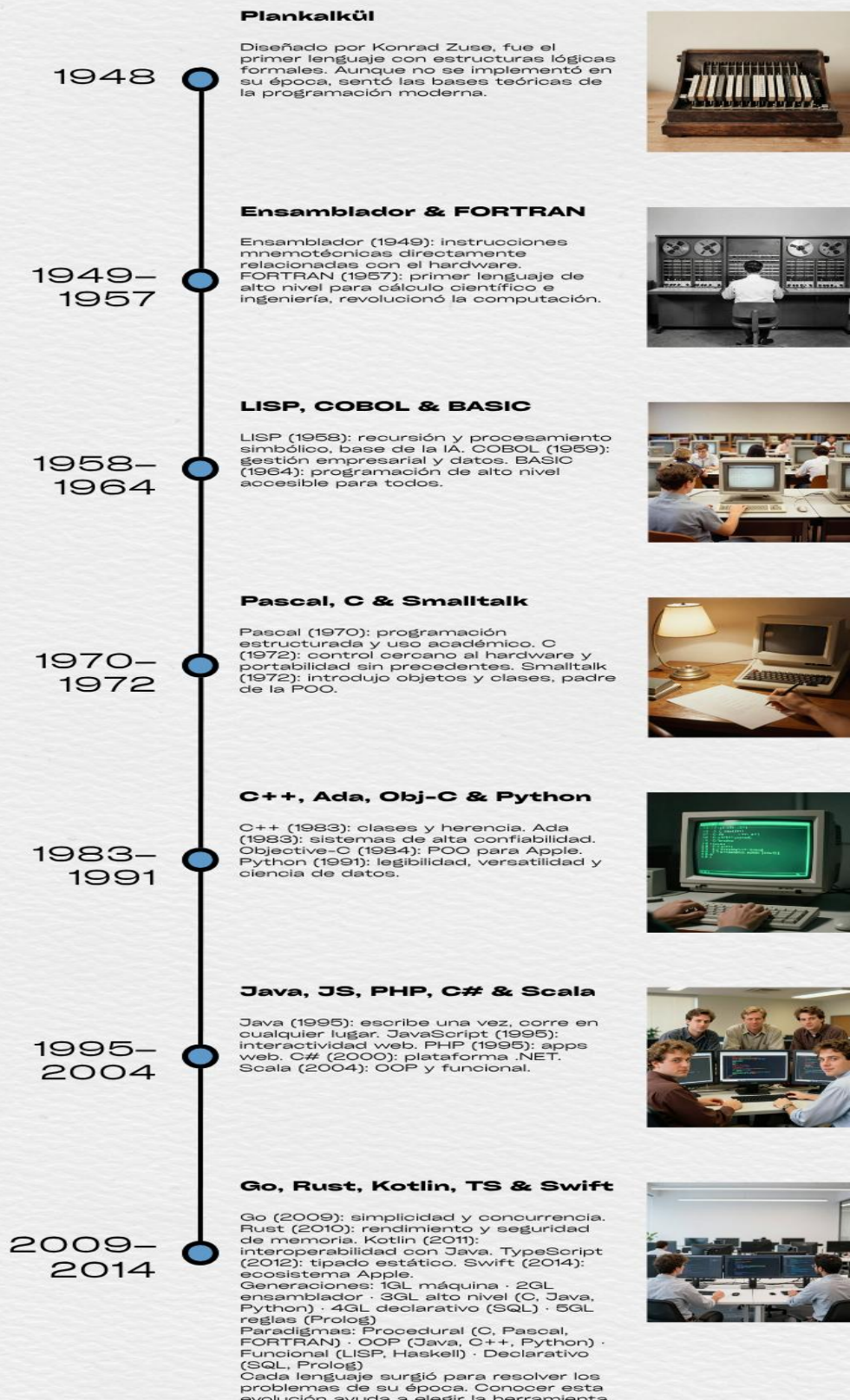
Ejemplos: SQL y Prolog.

El programador especifica **qué resultado desea obtener**, sin tener que definir detalladamente todos los pasos necesarios para conseguirlo. El sistema se encarga de determinar cómo ejecutar la solicitud.

Conclusión: La evolución de los lenguajes de programación demuestra cómo la informática ha avanzado desde instrucciones directamente relacionadas con el hardware hasta lenguajes de alto nivel capaces de resolver problemas complejos.

Ocho décadas de innovación: del código máquina a los lenguajes modernos

HISTORIA Y EVOLUCIÓN DE LOS LENGUAJES DE PROGRAMACIÓN





2.7 Resultados obtenidos

Se comprendió la evolución de los lenguajes de programación, sus principales características, generaciones y paradigmas, desde el código máquina hasta los lenguajes modernos.

2.8 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☐ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☒ Pensamiento crítico
- ☐ Flexibilidad
- ☒ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

- Se comprendió que los lenguajes de programación han evolucionado desde el código máquina y el ensamblador hacia lenguajes de alto nivel, buscando facilitar la programación y mejorar la productividad del desarrollador.
- Se identificó que cada generación de lenguajes de programación ha incorporado nuevas características y niveles de abstracción para resolver problemas de mayor complejidad.
- Se reconoció la importancia de los diferentes paradigmas de programación, ya que cada uno proporciona distintas formas de organizar y resolver problemas mediante código.

2.10 Recomendaciones

- Continuar investigando la evolución de los lenguajes de programación para comprender mejor el origen de las herramientas utilizadas actualmente.
- Practicar diferentes paradigmas de programación para desarrollar una mejor capacidad de análisis y selección de soluciones según el problema planteado.
- Utilizar fuentes confiables y herramientas tecnológicas de apoyo para complementar el aprendizaje y mantenerse actualizado sobre los lenguajes de programación modernos.

2.11 Referencias bibliográficas

Sebesta, R. W. (2019). *Concepts of programming languages* (12th ed.). Pearson.

Sebesta, R. W., Mukherjee, S., & Bhattacharjee, A. K. (2016). *Concepts of programming languages*. Pearson.

2.12 Anexos

sebesta, r. w. (2019). concepts of programming languages (12th ed.). pearson

Artículos

Cualquier momento

Desde 2026

Desde 2025

Desde 2022

Intervalo específico...

Ordenar por relevancia

Ordenar por fecha

[LIBRO] Concepts of programming languages

RW Sebesta, S Mukherjee, AK Bhattacharjee - 1999 - pdfs.semanticscholar.org

... ■ Increase our capacity to use different constructs ■ Enable us to choose **languages** more intelligently ■ Makes learning new **languages** easier ... www.aw.com/sebesta - This site contains mini-manuals (approximately 100-page tutorials) on a handful of **languages**. Currently the site includes manuals for C++, C, Java, and Smalltalk. **Language** Processor Availability-Processors for and information about some of the **programming languages** discussed in this book can be found at the following Web sites: ■ C, C++, Fortran, and Ada ...

☆ Guardar 📄 Citar Citado por 1373 Artículos relacionados Las 29 versiones 🔍